

Path-Coloring Algorithms for Plane Graphs

Aven Bross

Department of Computer Science

University of Alaska

Fairbanks, AK 99775-6670

dabross@alaska.edu

Glenn G. Chappell

Department of Computer Science

University of Alaska

Fairbanks, AK 99775-6670

chappellg@member.ams.org

Chris Hartman

Department of Computer Science

University of Alaska

Fairbanks, AK 99775-6670

cmhartman@alaska.edu

February 20, 2025

2010 Mathematics Subject Classification. Primary 05C38; Secondary 05C10, 05C15.

Key words and phrases. Path coloring, list coloring, algorithm.

Abstract

A path coloring of a graph G is a vertex coloring of G such that each color class induces a disjoint union of paths. We present two efficient algorithms to construct a path coloring of a plane graph.

The first algorithm, based on a proof of Poh, is given a plane graph, and produces a path coloring of the given graph using three colors.

The second algorithm, based on similar proofs by Hartman and Škrekovski, performs a list-coloring generalization of the above. The algorithm is given a plane graph and an assignment of lists of three colors to each vertex, and it produces a path coloring of the given graph in which each vertex receives a color from its list.

Implementations are available for all algorithms that are described.

1 Introduction

All graphs will be finite and simple. See West [12] for graph theoretic terms.

A graph is *planar* if it may be drawn in the plane by mapping each vertex to a point, and each edge to a simple arc between its incident vertices, such that the arcs intersect only at their endpoints. A *face* is a region of the plane not containing any point of the

drawing; we'll refer to each face by the subgraph drawn on its border. A (*combinatorial*) *embedding* is an equivalence class of drawings determined by the circular ordering of edges around each vertex. A *plane graph* is a planar graph with a fixed embedding. Subgraphs of a plane graph are plane graphs with the inherited embedding.

A *path coloring* is a (vertex) coloring where each color class induces a disjoint union of paths. A graph G is *path k -colorable* if G admits a path k -coloring. A *list assignment* L of a graph G maps each $v \in G$ to a finite set of colors $L(v)$. An *L -coloring* (or *L -list-coloring*) is a coloring c such that $c(v) \in L(v)$ for each $v \in V(G)$. A graph is *path k -choosable* if it admits a path L -coloring for every list assignment L with at least k colors in each list, i.e. $|L(v)| \geq k$ for each $v \in G$.

Broere & Mynhardt conjectured [2, Conj. 16] that every planar graph is path 3-colorable. This was proven independently by Poh [9, Thm. 2] and Goddard [6, Thm. 1].

Theorem 1.1 (Poh 1990, Goddard 1991). *If G is a planar graph, then G is path 3-colorable.* $\square$

It is easily shown that the “3” in Theorem 1.1 is best possible. In particular, Chartrand & Kronk [5, Section 3] gave an example of a planar graph whose vertex set cannot be partitioned into two subsets, each inducing a forest.

Hartman [7, Thm. 4.1] proved a list-coloring generalization of Theorem 1.1 (see also Chappell & Hartman [4, Thm. 2.1]). Essentially the same coloring technique was used by Škrekovski [11, Thm. 2.2b] to prove a result slightly weaker than Theorem 1.2.

Theorem 1.2 (Hartman 1997). *If G is a planar graph, then G is path 3-choosable.* $\square$

We discuss two path coloring algorithms based on proofs of the above theorems.

In Section 2 we outline our graph representations and the basis for our computations of time complexity.

Section 3 covers an algorithm based on Poh's proof of Theorem 1.1. The algorithm is given a plane graph; it produces a path coloring of the given graph using three colors.

Section 4 covers an algorithm based on Hartman's proof of Theorem 1.2, along with the proof of Škrekovski mentioned above. The algorithm is given a plane graph and an assignment of a list of three colors to each vertex; it produces a path coloring of the given graph in which each vertex receives a color from its list.

Section 5 provides benchmark results for implementations of each algorithm [3]. The section also discusses how each algorithm may be modified to benefit from parallelism.

2 Graph Representations and Time Complexity

A plane graph G is *triangulated* (or *maximal*) if it is connected and $n(G) \leq 2$, or if every face of G is a *triangle* (a length 3 cycle). We say G is *weakly triangulated* if all of its interior (bounded) faces are triangles (a triangulated graph is also weakly triangulated). Note that if G is biconnected and $n(G) \geq 3$, then every face of G is a cycle.

The input for each algorithm will be a biconnected, weakly triangulated plane graph G with n vertices and m edges. The input size will be n . By Euler's formula $m \leq 3n - 6$, so

it is equivalent use input size m . Planar graphs may be plane embedded and triangulated (by adding edges) in $\mathcal{O}(n)$ time, see e.g. Hopcroft and Tarjan [8] or Boyer and Myrvold [1].

We will represent G for computation via *adjacency lists*: a list, for each vertex v , of the neighbors of v . A vertex is represented by an integer $0 \dots n - 1$. The order of each list represents the embedding. In algorithms $N(v)$ will refer to the adjacency list.

Let $v \in G$ and $u \in N(v)$. Define $\text{next}_v(u) \in N(v)$ to be the neighbor counter-clockwise from u around v , and $\text{prev}_v(u) \in N(v)$ to be the neighbor clockwise from u . In algorithms ${}^v u$ will be a reference to u 's entry in $N(v)$, i.e. ${}^v u$ represents both u and the position of u in v 's adjacency list; define $\text{next}({}^v u) = {}^v(\text{next}_v(u))$ and $\text{prev}({}^v u) = {}^v(\text{prev}_v(u))$.

Adjacency lists naturally allow $\text{next}({}^v u)$ to be computed in constant time. In Section 4 however, the linear time complexity of the algorithm discussed depends on being able to compute $\text{prev}({}^v u)$ and the *back reference* ${}^u v$ from ${}^v u$ in constant time as well. Representing $N(v)$ via an array or doubly linked list solves computing both $\text{next}({}^v u)$ and $\text{prev}({}^v u)$, but back references require more information. We define an *augmented adjacency list* to be an adjacency list that stores the pair $(u, {}^u v)$ for each neighbor u of v . Given an adjacency list representation, an augmented representation may be constructed in $\mathcal{O}(m)$ time.

Algorithm 2.1: Augment Adjacency Lists

Input. An adjacency list representation N of a graph G .

Output. An augmented adjacency list representation N^* of G with the same neighbor ordering as N .

Step 1. For each $v \in G$ construct an augmented adjacency list $N^*(v)$ with the same ordering as $N(v)$, leaving the ${}^u v$ portion of each entry $(u, {}^u v)$ in $N^*(v)$ uninitialized.

Step 2. For each vertex v construct a list $\text{Wrk}(v)$ of vertex-reference pairs with capacity $\deg(v)$. For each v from 0 to $n - 1$ iterate through $N^*(v)$. For each neighbor u in $N^*(v)$ append the pair $(v, {}^v u)$ to $\text{Wrk}(u)$, where ${}^v u$ is a reference to u 's entry $N^*(v)$.

Step 3. For each v from $n - 1$ to 0 iterate through $\text{Wrk}(v)$. Upon reaching a pair $(u, {}^u v)$ in $\text{Wrk}(v)$ the last element of $\text{Wrk}(u)$ will be $(v, {}^v u)$; see the below. Lookup ${}^v u$ in $N^*(v)$ and initialize the back reference ${}^u v$, and vice versa. Pop $(v, {}^v u)$ from $\text{Wrk}(u)$.

After completing Step 2 in Algorithm 2.1 each list $\text{Wrk}(v)$ will contain $(u, {}^u v)$ for each neighbor u of v , sorted in increasing order by u . Suppose that v is the current vertex at a given iteration of the loop in Step 3. For each edge $uw \in E(G)$ such that $u < w$ and $v < w$, prior iterations of the loop will have initialized the back references for $(w, {}^w u)$ in $N^*(u)$ and $(u, {}^u w)$ in $N^*(w)$, and removed $(w, {}^w u)$ from $\text{Wrk}(u)$. Therefore for each $(u, {}^u v)$ in $\text{Wrk}(v)$, the list $\text{Wrk}(u)$ will only contain entries $(w, {}^w u)$ where $w \leq v$.

3 Path Coloring

In this section we describe a linear time algorithm to path 3-color plane graphs. Let's first recount Poh's path 3-coloring strategy.

Let C be a cycle in a plane graph G . If $u, v \in C$, then we define $C[u, v]$ to be the clockwise u, v -path around C ; if $u = v$, then $C[u, u] = u$. Define $\text{int}(C)$ to be the subgraph

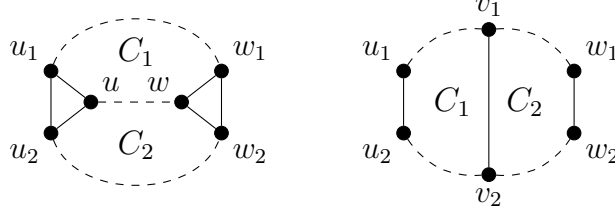


Figure 1: The proof of Lemma 3.1 when C is induced (left) or has a chord (right).

of G consisting of C and all vertices and edges interior to C ; equivalently, $\overline{\text{int}}(C)$ is the maximal subgraph of G with outer face C . Define $\text{int}(C) = \overline{\text{int}}(C) - C$.

Let P be a path in a plane graph G . If $u, v \in P$, then we define $P[u, v]$ to be the u, v -path in P . We also define $\overline{\text{int}}(P) = P$ and $\text{int}(P)$ to be empty.

Lemma 3.1 (Poh 1990). *Let G be a biconnected, weakly triangulated plane graph with outer face C . Let $c : V(C) \rightarrow S \subsetneq \{1, 2, 3\}$ be a 2-coloring of C such that each color class induces a nonempty path. There exists an extension of c to a path 3-coloring $c : V(G) \rightarrow \{1, 2, 3\}$ such that for each $v \in G - C$, if $vu \in E(G)$ with $u \in C$, then $c(v) \neq c(u)$.*

Proof. If $n(G) \leq 3$, then $G = C$. We proceed by induction on the order of G .

Let P_1, P_2 be the two paths induced by the 2-coloring of the outer face C . We may write $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$ for vertices $u_1, u_2, w_1, w_2 \in C$.

Suppose that C is an induced subgraph of G . Let $u, w \in G - C$ be the vertices such that the cycles u, u_2, u_1 and w, w_1, w_2 are faces of G ; note that u and w are uniquely determined, but it may be that $u = w$. Since C is induced, $G - C$ is connected. Let P_3 be a u, w -path in $G - C$ of minimal length, and note that P_3 is an induced subgraph of G . Color each vertex of P_3 with the color $d \in \{1, 2, 3\} - S$. Let $C_1 = P_1 \cup P_3 \cup uu_1 \cup ww_1$ and $C_2 = P_2 \cup P_3 \cup uu_2 \cup ww_2$, see Figure 1. By the inductive hypothesis, c extends to path 3-colorings c_1 of $\overline{\text{int}}(C_1)$ and c_2 of $\overline{\text{int}}(C_2)$. Together c_1 and c_2 form an extension of c to a 3-coloring of G . Since $C, P_3, \text{int}(C_1), \text{int}(C_2)$ partition $V(G)$, and no vertex in one partition shares a color with a neighbor in another, this extension is a path coloring.

Suppose that C is not an induced subgraph. Then there exists an edge $v_1v_2 \in E(G) - E(C)$ with $v_1 \in P_1$ and $v_2 \in P_2$. Let $C_1 = C[v_2, v_1] \cup v_1v_2$ and $C_2 = C[v_1, v_2] \cup v_1v_2$, see Figure 1. By the inductive hypothesis, c extends to path 3-colorings of $\overline{\text{int}}(C_1)$ and $\overline{\text{int}}(C_2)$. Together these colorings form a path 3-coloring of G . $\square$

Let G be a triangulated plane graph. We may trivially path 2-color the outer triangle. By Lemma 3.1 this extends to a path 3-coloring of G . If G is an arbitrary planar graph, we may compute an embedding and add edges to produce a triangulated plane graph G' . Theorem 1.1 follows immediately as any path coloring of G' is also a path coloring of G .

A natural way to implement Poh's algorithm is to locate u , and then use a breadth-first search to construct a minimal u, w -path and/or locate a chord edge, see Algorithm 3.2.

Algorithm 3.2: Path 3-Color (BFS)

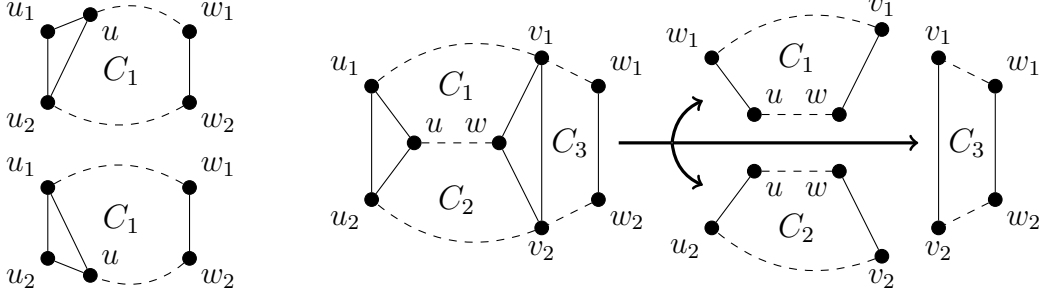


Figure 2: Algorithm 3.2, Case 1 (left) and Case 2 (right).

Input. Let C be a cycle or length 1 path in a weakly triangulated plane graph G represented via adjacency lists. Let c be a length n array representing a 2-coloring of C such that each color class induces a nonempty path, respectively $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$ for $u_1, u_2, w_1, w_2 \in C$. Assume $c(v) = 0$ for each $v \in \text{int}(C)$.

The concrete input will be G, c , and the tuple $(u_1, w_1, {}^{u_1}u_2, w_2)$.

Output. For each vertex $v \in \text{int}(C)$, a color in $\{1, 2, 3\}$ will be assigned to $c(v)$ such that c represents a path 3-coloring of $\overline{\text{int}}(C)$ extending the original 2-coloring of C . Moreover, if $v \in \text{int}(C)$ has a neighbor $u \in C$, then $c(v) \neq c(u)$.

Base case. If $u_1 = w_1$ and $u_2 = w_2$, then C is a length 1 path and $\overline{\text{int}}(C) = C$.

Recursive case. Let ${}^{u_1}u \leftarrow \text{next}({}^{u_1}u_2)$. Note that u, u_2, u_1 is a face of G , see Figure 2.

Case 1. Suppose that $c(u) \neq 0$. If $c(u) = c(u_2)$, then $u \in P_2$. Make a recursive call with the input $(u_1, w_1, {}^{u_1}u, w_2)$ to color $\overline{\text{int}}(C_1)$ where $C_1 = C[u_1, u] \cup u_1u$. Otherwise, $c(u) = c(v_1)$ and $u \in P_1$. Iterate through $N(u)$ to find ${}^u u_2$. Make a recursive call with the input $(u, w_1, {}^u u_2, w_2)$ to color $\overline{\text{int}}(C_1)$, where $C_1 = C[u, u_2] \cup uu_2$.

Case 2. Suppose that $c(u) = 0$, i.e. $u \in \text{int}(C)$. Perform a breadth-first search in $\text{int}(C)$ starting from u ; such a search may be implemented by visiting $v \in G$ only if $c(v) = 0$. Terminate the search upon finding a vertex w with a pair of subsequent neighbors ${}^w v_2$ and ${}^w v_1 = \text{next}({}^w v_2)$ such that $c(v_2) = c(u_2)$ and $c(v_1) = c(u_1)$, i.e. $v_2 \in P_2$ and $v_1 \in P_1$. Backtrack along the search tree from w to find a minimal u, w -path P_3 . For each vertex $v \in P_3$, assign $c(v) \leftarrow d$, where $d \in \{1, 2, 3\} - \{c(u_1), c(u_2)\}$.

Iterate through $N(u)$ and $N(v_1)$ to find ${}^u u_2$ and ${}^{v_1} v_2$. Finally, to color the remaining vertices in $\text{int}(C) - P_3$, we make a recursive call

1. with input $(u_1, v_1, {}^{u_1}u, w)$ to color $\overline{\text{int}}(C_1)$, where $C_1 = C[u_1, v_1] \cup P_3 \cup v_1u \cup v_1w$,
2. with input $(u, w, {}^u u_2, v_2)$ to color $\overline{\text{int}}(C_2)$, where $C_2 = C[v_2, u_2] \cup P_3 \cup uu_2 \cup ww_2$,
3. and with input $(v_1, w_1, {}^{v_1} v_2, w_2)$ to color $\overline{\text{int}}(C_3)$, where $C_3 = C[v_1, v_2] \cup v_1v_2$.

Let G be a triangulated plane graph and let $C = v_1, v_2, v_3$ be the outer triangle of G labeled in clockwise order. We may compute a path 3-coloring c of G as follows. Create a length n integer array c and initialize each entry to zero. Assign $c(v_1) \leftarrow 1$, $c(v_2) \leftarrow 2$, and $c(v_3) \leftarrow 2$. Finally, apply Algorithm 3.2 with input G, c , and $(v_1, v_1, {}^{v_1} v_2, v_3)$.

Unfortunately, Algorithm 3.2 is not linear. Consider the sequence of “pyramid graphs” $\{A_k\}_{k \in \mathbb{N}}$ depicted in Figure 3. Fix $k \in \mathbb{N}$ and note that $n = n(A_k) = k(k+1)/2 + 3$.

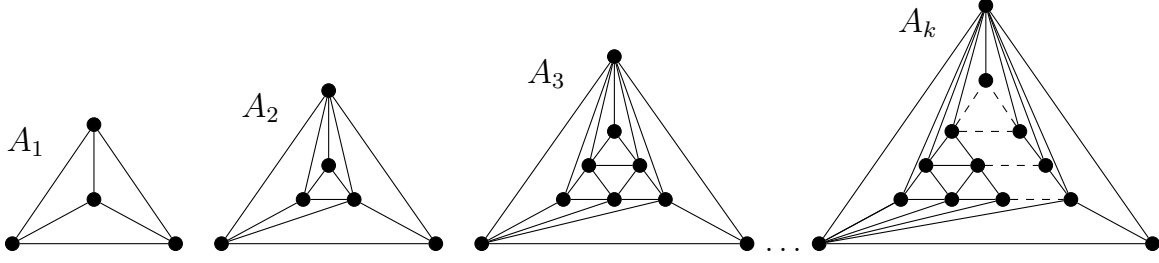


Figure 3: The sequence of “pyramid graphs” on which Algorithm 3.2 performs poorly.

Assume that the outer triangle is 2-colored such that the top vertex is assigned a color distinct from the bottom two. At recursive depth i , Algorithm 3.2 will perform a breadth first search of the pyramid shaped interior subgraph with sides of length $r = k - i$, starting from the vertex in the bottom left corner. The search will hit all $r(r + 1)/2$ vertices of the interior pyramid. Therefore the total number of operations performed will be

$$\Omega\left(\sum_{r=1}^k \frac{r(r+1)}{2}\right) = \Omega(k^3) = \Omega(n^{3/2}).$$

However, Poh’s argument only relied on P_3 being an induced u, w -path, not a path of minimal length. Algorithm 3.3 describes a linear time implementation of Poh’s algorithm.

For a subgraph H of G , let $N(H)$ be the set of vertices in $G - H$ with a neighbor in H . Let C and $P_1 = C[u_1, w_1]$ be as in Lemma 3.1. Let $p_1 : N(P_1) \rightarrow V(P_1)$ map each $v \in N(v)$ to its neighbor farthest clockwise along P_1 , i.e. $V(C[p_1(v), w_1]) \cap N(v) = \{p_1(v)\}$.

Algorithm 3.3: Path 3-Color (Neighbor Trace)

Input. Let C be an induced cycle in a weakly triangulated plane graph G represented via adjacency lists. Let c be a length n array representing a 2-coloring of C such that each color class induces a nonempty path, respectively $P_1 = C[u_1, w_1]$ and $P_2 = C[w_2, u_2]$, where $u_1, u_2, w_1, w_2 \in C$. Assume that $c(v) = 0$ for each $v \in \text{int}(C)$. Let S be a length n integer array and let ℓ_1 be an integer such that for each $v \in \text{int}(C)$ we have $S(v) = \ell_1$ if and only if $v \in N(P_1)$.

The concrete input will be G, c , and S , along with the tuple (u_1, u_2, ℓ_1) .

Output. For each vertex $v \in \text{int}(C)$ a color in $\{1, 2, 3\}$ will be assigned to $c(v)$ such that c represents a path 3-coloring of $\overline{\text{int}(C)}$ extending the original 2-coloring of C . Moreover, if $v \in \text{int}(C)$ has a neighbor $u \in C$, then $c(v) \neq c(u)$.

Procedure. Assign $u_1 u \leftarrow \text{next}(u_1 u_2)$.

Base case. If $c(u) \neq 0$, then $u = w_2$ or $u = w_1$. Thus C is a face and we are done.

Recursive case. Suppose that $c(u) = 0$, i.e. $u \in \text{int}(C)$. Let $d \in \{1, 2, 3\} - \{c(u_1), c(u_2)\}$. Let $\ell_3 = \max_{v \in G} S(v) + 1$. We will construct an induced u, w -path $P_3 = v_1, v_2, \dots, v_j$ such that $v_1, v_2, \dots, v_j \in N(P_1)$, and color each $v \in P_3$ with $c(v) \leftarrow d$. We’ll also mark each $v \in N(P_3) - V(C)$ with $S(v) \leftarrow \ell_3$, and make recursive calls to color $\text{int}(C) - P_3$. It may be useful to refer to Figures 4 and 5.

Step 1. Assign $j \leftarrow 1$ and $v_j \leftarrow u$; here j, v_j store the end of the path P_3 that we've constructed so far. Set $v_0 \leftarrow u_2$ so that v_{j-1} is defined for $j = 1$. Assign $i \leftarrow 1$ and $v_i s \leftarrow v_i u_2$; here $i, v_i s$ store the last edge $v_i s$ that was visited where $v_i \in P_3$ and $s \in P_2$.

Loop 2. We'll build the path P_3 , adding one vertex per iteration. Assign $c(v_j) \leftarrow d$. Find $v_j v_{j-1}$ in $N(v_j)$ and set $v_j v \leftarrow \text{next}(v_j v_{j-1})$. Set $y \leftarrow \text{NULL}$; here y will store the first neighbor of v_j counter-clockwise from v_{j-1} that is in $N(P_1) - V(C)$ (if any). Set $v_j t \leftarrow \text{NULL}$; here $v_j t$ will store the last edge $v_j t$ encountered such that $t \in P_1$ (if any).

Loop 2.1. We'll first consider the neighbors of v_j between v_{j-1} and $p_1(v_j)$ counter-clockwise. Each iteration will handle the current neighbor $v_j v$. We'll make a recursive call for each induced cycle C_1 we discover with $V(C_1) \subseteq V(P_2) \cup V(P_3)$. Refer to Figure 4.

Case 2.1.1. Suppose that $c(v) = 0$, i.e. $v \in \text{int}(C)$. If $S(v) = \ell_1$, i.e. $v \in N(P_1)$, and $y = \text{NULL}$, then set $y \leftarrow v$. Set $S(v) \leftarrow \ell_3$ and $v_j v \leftarrow \text{next}(v_j v)$. Repeat Loop 2.1.

Case 2.1.2. Suppose that $c(v) = c(u_2)$, i.e. $v \in P_2$. Make a recursive call with input $(v_i, v_i s, \ell_3)$ to color $\overline{\text{int}}(C_1)$, where $C_1 = P_3[v_i, v_j] \cup C[v, s] \cup v_i s \cup v_j v$; see Figure 4 (middle). Assign $i \leftarrow j$, $v_i s \leftarrow v_j v$, and $v_j v \leftarrow \text{next}(v_j v)$. Repeat Loop 2.1.

Case 2.1.3. Suppose that $c(v) = c(u_1)$, i.e. $v = p_1(v_j) \in P_1$. Assign $v_j t \leftarrow v_j v$ and $v_j v \leftarrow \text{next}(v_j v)$. Break Loop 2.1 and proceed to Loop 2.2.

Note that if $y = \text{NULL}$, then $v_j = w$. To see this, observe that since $v \in P_1$, $\text{prev}_{v_j}(v) \in N(P_1)$, and $\text{prev}_{v_j}(v) \notin P_1$, we must have $\text{prev}_{v_j}(v) \in C - P_1 = P_2$.

Loop 2.2. We'll now consider the neighbors of v_j between $p_1(v_j)$ and v_{j-1} counter-clockwise. Each iteration will handle the current neighbor $v_j v$. We'll make a recursive call for each induced cycle C_1 we discover with $V(C_1) \subseteq V(P_1) \cup V(P_3)$. Observe $v_j t \neq \text{NULL}$.

Case 2.2.1. Suppose that $c(v) = 0$, i.e. $v \in \text{int}(C) - P_3$. Assign $S(v) \leftarrow \ell_3$ and $v_j v \leftarrow \text{next}(v_j v)$. Repeat Loop 2.2.

Case 2.2.2. Suppose that $c(v) \neq 0$, i.e. $v \in C$ or $v \in P_3$. Make a recursive call with input $(v_j, v_j t, \ell_3)$ to color $\overline{\text{int}}(C_1)$, where both the definition of C_1 and the next steps to take depend on $c(v)$. Refer to Figure 5.

If $c(v) = c(u_1)$, i.e. $v \in P_1$, then $C_1 = C[v, t] \cup v_j \cup v_j t \cup v_j v$. Assign $v_j t \leftarrow v_j v$ and $v_j v \leftarrow \text{next}(v_j v)$. Repeat Loop 2.2.

Otherwise, $v = v_{j-1}$ (either $j > 1$ and $v \in P_3$, or $j = 1$ so $v = u_2 \in P_2$ and $t = u_1$). Then $C_1 = C[p_1(v), t] \cup P_3[v, v_j] \cup v p_1(v) \cup v_j t$. Break Loop 2.2 and proceed to Step 2.3.

Step 2.3. If $y \neq \text{NULL}$, then assign $j \leftarrow j + 1$, $v_j \leftarrow y$, and repeat Loop 2. Otherwise, $v_j = w$ and we are done.

We claim that the path $P_3 = v_1, v_2, \dots, v_j$ in Algorithm 3.3 is always an induced subgraph of G . Suppose that $v_k v_j \in E(G)$ with $0 < k < j - 1$. Observe that

$$D = C[u_2, p_1(v_{j-1})] \cup P_3[v_1, v_{j-1}] \cup u_2 v_1 \cup v_{j-1} p_1(v_{j-1})$$

is a cycle in G with $v_{k-1}, v_k, v_{k+1} \in D$ and $v_j \notin \overline{\text{int}}(D)$, see Figure 5 (right). Therefore $v_k v_j \notin \overline{\text{int}}(D)$ and $v_k v_j$ lies between v_{k-1} and v_{k+1} counter-clockwise in $N(v_k)$. But v_{k+1} is the first neighbor in $N(P_1)$ counter-clockwise from v_{k-1} around v_k , a contradiction.

Given a triangulated plane graph G represented via adjacency lists, we can apply Algorithm 3.3 as follows. First, create length n integer arrays for c and S , initializing

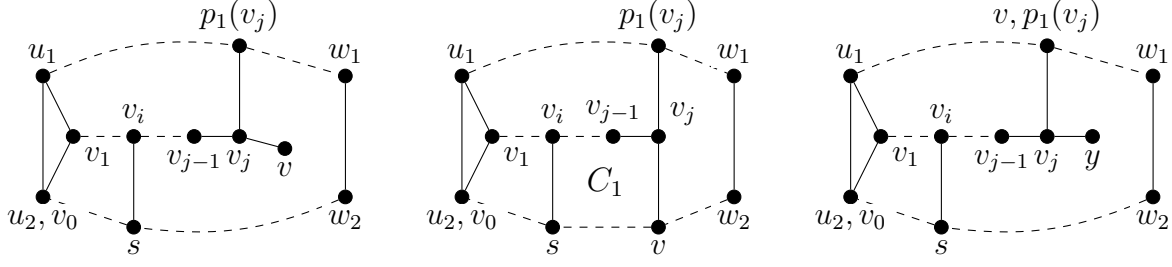


Figure 4: Algorithm 3.3, Case 2.1.1 (left), Case 2.1.2 (middle), and Case 2.1.3 (right).

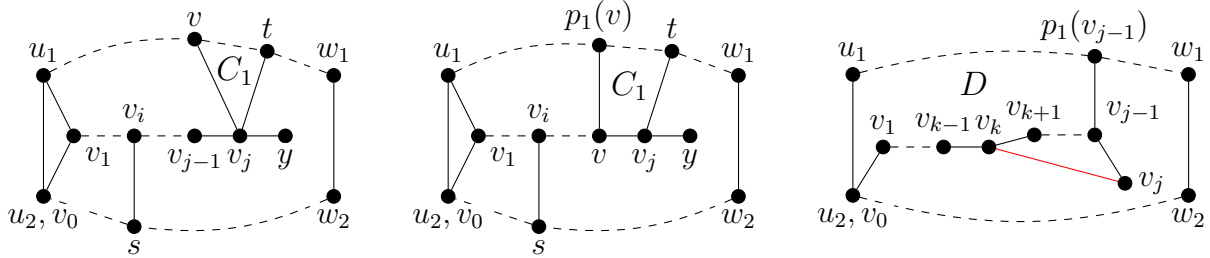


Figure 5: Algorithm 3.3, Case 2.2.2 when $v \in P_1$ (left) and when $v = v_{j-1}$ (middle). Also, a sketch of why P_3 is an induced path (right).

each entry to zero. Let $C = v_1, v_2, v_3$ be the outer triangle of G , labeled in clockwise order. Color $c(v_1) \leftarrow 1$, $c(v_2) \leftarrow 2$, and $c(v_3) \leftarrow 2$. Iterate through $N(v_1)$ and mark $S(v) \leftarrow 1$ for each neighbor v of v_1 . Then $G, c, S, (v_1, v_1 v_3, 1)$ form a valid input for Algorithm 3.3.

During Algorithm 3.3 we iterate through $N(v)$ for each $v \in G$ exactly twice: once to locate $v_j v_{j-1}$ when $v_j = v$ is colored and added to a path P_3 in Loop 2, and once to iterate through $N(v_j)$ in Loops 2.1 and 2.2. Therefore the time complexity is $\mathcal{O}(n)$.

4 Path List-Coloring

In this section we describe a linear time algorithm to path color plane graphs such that each vertex receives a color from a specified list. Hartman showed that this is always possible when each vertex is given a list of 3 colors [7, Thm. 4.1]. Around the same time Škrekovski proved a weaker result using the same coloring strategy [11, Thm. 2.2b].

In brief, the Hartman-Škrekovski algorithm takes as input a weakly triangulated plane graph G with outer face C , a list assignment L , and two vertices $x, y \in C$. It assumes that $|L(x)| \geq 1$, $|L(y)| \geq 1$, $|L(v)| \geq 2$ for each $v \in C - \{x, y\}$, and $|L(v)| \geq 3$ for each $v \in G - C$. A color $d \in L(x)$ is selected, and an induced path P is colored with d , starting from x , and proceeding clockwise towards y along $C[x, y]$, such that for each $v \in P$, $d \in L(v)$. The algorithm is then applied to each connected component of $G - P$, using a new list assignment L' defined by $L'(v) = L(v) - \{d\}$ if $v \in N(P)$, $L'(v) = L(v)$

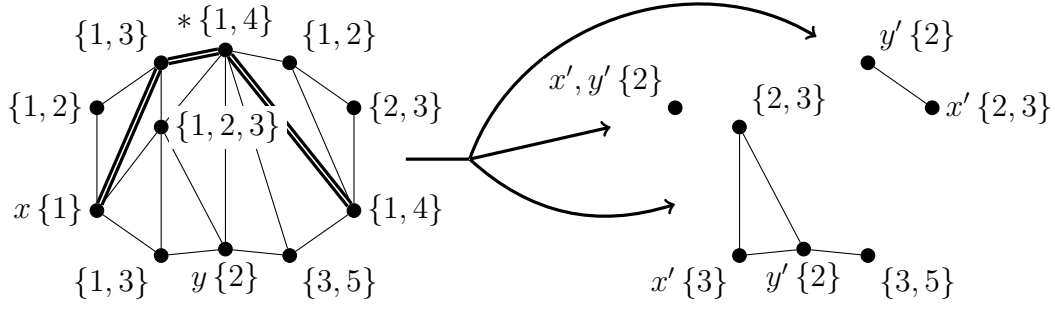


Figure 6: Coloring and removing a path P in Hartman's proof [7, Lemma 4.2].

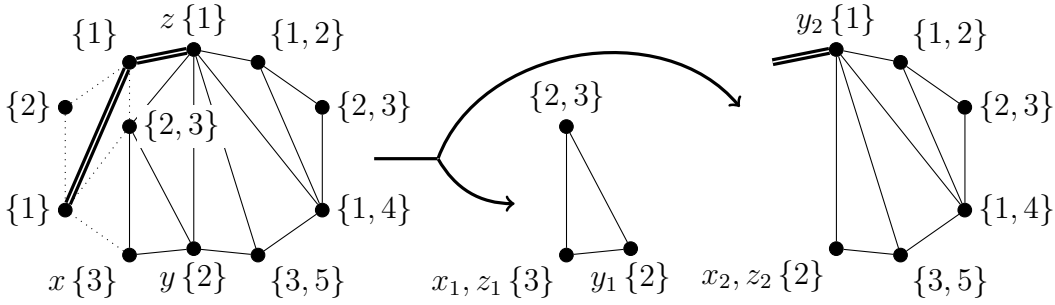


Figure 7: Coloring and removing a path P in the proof of Lemma 4.1 (Algorithm 4.2).

if $v \notin N(P)$. This produces a path L -coloring of G . See Figure 6 for an example.

In Lemma 4.1 below, we provide a different proof of Hartman's result, organized in such a way that it immediately describes a linear time algorithm for graphs represented as in Section 2. If $x = z$, then our G, x, y match those in Hartman's [7, Lemma 4.2], but we assume that G is biconnected; we follow a similar iterative procedure to color a path P clockwise from x along C , but we consider one edge at a time, and simultaneously remove P as it is colored. If $x \neq z$, then z is the farthest clockwise end of the path P being colored; the rest of P has already been removed (our x is now Hartman's x').

For those familiar with Hartman's proof: the one major difference between our algorithm and Hartman's occurs in our Case 3.2.3. If we encounter a chord edge $zv \in G - C$ (z is the current end of P) with $v \in C[z, y]$ and $L(z) \cap L(v) = \emptyset$ (i.e. $d \notin L(v)$, where d is the color of the path P), instead of continuing to search counter-clockwise for neighbors of z in $C[z, v]$ that we might add to P (as in the proof of [7, Lemma 4.2]), we apply the inductive hypothesis such that z now serves as "y" (see Figure 10 for an illustration of this case in general). The chord from the vertex labeled $*$ to y in Figure 6 is an example of such an edge: our algorithm eventually produces the same coloring as Hartman's on this input G, L, x, y , but it initially halts P at $*$, and separately considers the left and right halves of the graph, with $*$ as the new "y" on the right, see Figure 7.

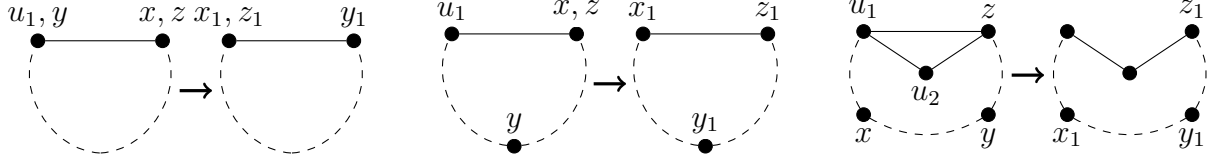


Figure 8: The proof of Lemma 4.1, Case 1 (left), Case 2 (middle), and Case 3.1 (right).

Given a coloring c of a graph G , and $v \in G$, we define $\deg_c(v)$ to be $\deg(v)$ in the subgraph induced by $c(v)$'s color class, i.e. the number of neighbors that share v 's color.

Lemma 4.1. *Let G be a biconnected, weakly triangulated plane graph with outer face C . Let $x, y, z \in C$ be vertices (not necessarily distinct) such that $z \in C[x, y]$. Let L be a list assignment of G such that*

$$\begin{aligned} |L(v)| &= 1 \text{ for } v \in \{x, y, z\}, \\ |L(v)| &\geq 2 \text{ for } v \in C - \{x, y, z\}, \\ |L(v)| &\geq 3 \text{ for } v \in G - C, \end{aligned}$$

and if $v \in C[x, z] - z$, then $L(v) \cap L(z) = \emptyset$. There exists a path L -coloring of G such that $\deg_c(x) \leq 1$, $\deg_c(y) \leq 1$, and $\deg_c(z) \leq 1$. Moreover, if $z = y$, or if $C[z, y] = z, y$ and $L(z) \cap L(y) = \emptyset$, then $\deg_c(z) = 0$.

Proof. Suppose that $m = |E(G)| \leq 1$. Then $G = C$. We define a path L -coloring c of G as follows. Define $c(x) \in L(x)$ and $c(y) \in L(y)$ to be the color in each respective list. If $m = 0$, then $n(G) = 1$ and c is trivially a path L -coloring. Otherwise $m = 1$ and $n(G) = 2$. If $x \neq y$, then c is a path L -coloring of G with the constraints of the lemma. Otherwise the remaining $v \in G - y$ satisfies $|L(v)| \geq 2$, and we define $c(v) \in L(v) - \{c(y)\}$.

We proceed by induction on m , the number of edges. Let $u_1, u_2, \dots, u_k$ label the neighbors of z in counter-clockwise order such that u_1 is the vertex immediately counter-clockwise from z around C , and u_k is the vertex clockwise from z around C . In each of the subsequent cases we will apply the lemma separately to one or two subgraphs G_1, G_2 of G , with corresponding labeled outer face vertices $x_i, y_i, z_i \in G_i$ for $i = 1, 2$.

Case 1. Suppose that $u_1 = y$. Note that $z = x$ since $z \in C[x, y]$. Therefore $V(C[z, y]) = V(C)$ and $|V(C[z, y])| \geq 3$, so $C[z, y] \neq z, y$. Apply the lemma to find a path L -coloring c of $G_1 = G$ with $x_1 = y$, $y_1 = x$, and $z_1 = y$, see Figure 8 (left).

Case 2. Suppose that $u_1 \neq y$ and $z = x$. Then $u_1 \in C - C[x, z]$ and $|L(u_1)| \geq 2$. Pick $d \in L(u_1) - L(z)$. Define $L'(u_1) = \{d\}$ and $L'(v) = L(v)$ for each $v \in G - u_1$. Apply the lemma to find a path L' -coloring of $G_1 = G$ with $x_1 = u_1$, $y_1 = y$, and $z_1 = z$, see Figure 8 (middle).

Case 3. Suppose that $u_1 \neq y$ and $z \neq x$. Our strategy will be to apply the inductive hypothesis to the *blocks* (maximal biconnected subgraphs) of $G - zu_1$. Because $u_1 \in C[x, z]$, we have $L(u_1) \cap L(z) = \emptyset$. Thus any path L -coloring of $G - zu_1$ is also a path L -coloring of G . There are three cases to consider, refer to Figures 8 (right), 9, and 10.

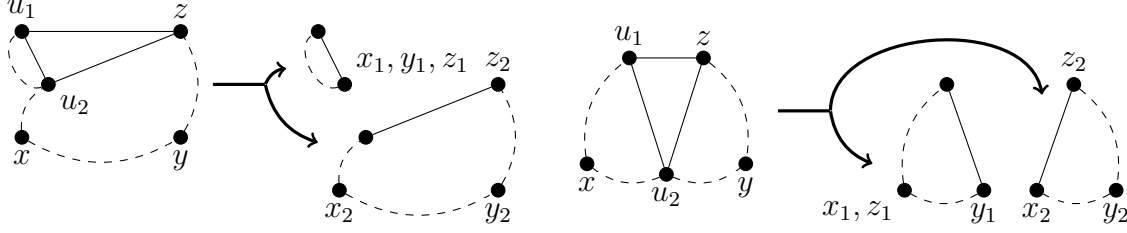


Figure 9: The proof of Lemma 4.1, Case 3.2.1 (left) and Case 3.2.2 (right).

Case 3.1. Suppose that $u_2 \in G - C$. Define $L'(u_2) = L(u_2) - L(z)$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Note that $|L'(u_2)| \geq |L(u_2)| - 1 \geq 2$. Let C_1 be the outer face of $G_1 = G - zu_1$, and observe that $C_1[x, z]$ is equal to $C[x, z]$ with the edge u_1z removed and the path u_1, u_2, z added. Since $|L'(u_2)| \geq 2$ and $L'(u_2) \cap L'(z) = \emptyset$, we may apply the inductive hypothesis to find a path L' -coloring of G_1 with $x_1 = x, y_1 = y, z_1 = z$.

Case 3.2. Suppose that $u_2 \in C$. Observe that u_2 is a *cut-vertex* of $G - zu_1$, i.e. $G - zu_1 - u_2$ is disconnected. Define $C_1 = C[u_2, u_1] \cup u_1u_2$ and $C_2 = C[z, u_2] \cup zu_2$. Let $G_1 = \text{int}(C_1)$ and $G_2 = \text{int}(C_2)$. The subgraphs G_1 and G_2 are the two blocks of $G - zu_1$. Note that if $k = 2$, then $G_1 = G - z$ and $G_2 = z, u_2$.

In each subsequent case we will apply the inductive hypothesis to find a path L -coloring c_1 of G_1 and a path L -coloring c_2 of G_2 such that $c_1(u_2) = c_2(u_2)$. Let c be the L -coloring of G defined by $v \mapsto c_1(v)$ for $v \in G_1$ and $v \mapsto c_2(v)$ for $v \in G_2$. Observe that $\deg_c(v) = \deg_{c_1}(v)$ for each $v \in G_1 - u_2$, $\deg_c(v) = \deg_{c_2}(v)$ for each $v \in G_2 - u_2$, and $\deg_c(u_2) = \deg_{c_1}(u_2) + \deg_{c_2}(u_2)$. To show that c is a path L -coloring of G it suffices to show that $\deg_c(u_2) \leq 2$. We'll also need to show that the $\deg_c$ constraints hold for x, y, z , but these often follow immediately from the inductive hypothesis.

Case 3.2.1. Suppose that $u_2 \in C[x, z]$. Observe that $x, y, z \in C_2$. Apply the inductive hypothesis to find a path L -coloring c_2 of G_2 with $x_2 = x, y_2 = y$, and $z_2 = z$. Define $L'(u_2) = \{c_2(u_2)\}$ and $L'(v) = L(v)$ for each $v \in G_1 - u_2$. Find a path L' -coloring c_1 of G_1 with $x_1 = y_1 = z_1 = u_2$. Since $\deg_{c_1}(u_2) = 0$, c is a path L -coloring of G . Moreover, c_2 , and hence c , satisfies the $\deg_c$ constraints of the lemma for x, y, z .

Case 3.2.2. Suppose that $u_2 \in C[y, x] - x - y$. Pick $d \in L(u_2) - L(z)$. Define $L'(u_2) = \{d\}$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Observe that $x \in C_1$ and $z, y \in C_2$. Apply the inductive hypothesis to find a path L' -coloring c_1 of G_1 with $x_1 = z_1 = x$ and $y_1 = u_2$. Then find a path L' -coloring c_2 of G_2 with $x_2 = u_2, y_2 = y$, and $z_2 = z$. Note that $\deg_{c_1}(u_2) \leq 1$ and $\deg_{c_2}(u_2) \leq 1$, thus c is a path L -coloring of G . Since $x_1 = x$ in $G_1 - u_2$, and $y_2 = y$ and $z_2 = z$ in $G_2 - u_2$, c satisfies the $\deg_c$ constraints of the lemma.

Case 3.2.3. Suppose that $u_2 \in C[z, y]$. Note that $z \neq y$. There are two cases to consider, see Figure 10.

Case 3.2.3.1. Suppose that $L(z) \cap L(u_2) \neq \emptyset$. Then either $u_2 = y$ and $L(z) = L(y)$, or $C[z, y] \neq z, y$. Define $L'(u_2) = L(z)$ and $L'(v) = L(v)$ for each $v \in G - u_2$. Find a path L' -coloring c_1 of G_1 with $x_1 = x, y_1 = y$, and $z_1 = u_2$. Then find a path L' -coloring c_2 of G_2 with $x_2 = u_2, y_2 = z$, and $z_2 = u_2$. Since $\deg_{c_1}(u_2) \leq 1$ and $\deg_{c_2}(u_2) = 1$, it

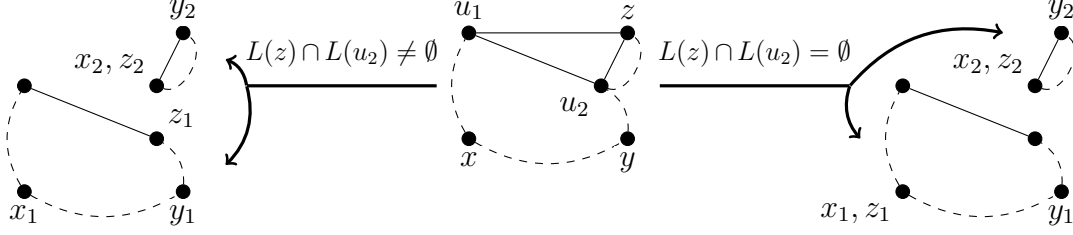


Figure 10: The proof of Lemma 4.1, Case 3.2.3.

follows that c is a path L -coloring of G . If $u_2 = y$, then $z_1 = y_1 = y$ and $\deg_{c_1}(y) = 0$; thus $\deg_c(y) = 1$. So $\deg_c(y) \leq 1$ and c satisfies the constraints of the lemma.

Case 3.2.3.2. Suppose that $L(z) \cap L(u_2) = \emptyset$. Find a path L -coloring c_1 of G_1 with $x_1 = x$, $y_1 = y$, and $z_1 = x$. Define $L'(u_2) = \{c_1(u_2)\}$ and $L'(v) = L(v)$ for each $v \in G_2 - u_2$. Then find a path L' -coloring c_2 of G_2 with $x_2 = u_2$, $y_2 = z$, and $z_2 = u_2$. Because $C[z_2, y_2] = z_2, y_2$ and $L'(z_2) \cap L'(y_2) = \emptyset$, we must have $\deg_{c_2}(u_2) = \deg_{c_2}(z_2) = 0$. Thus c is a path L -coloring of G .

Suppose $C[z, y] = z, y$. Then it must be that $u_2 = y$ and $k = 2$. Therefore $G_2 = z, y$ and $\deg_c(z) = 0$, since $c(z) \notin L(u_2)$. Thus c satisfies the constraints of the lemma. $\square$

Let G be a planar graph and L a list assignment such that $|L(v)| \geq 3$ for each $v \in G$. Compute a planar embedding and add edges to produce a triangulated plane graph G' . Pick $u \in G'$ on the outer face and $d \in L(u)$. Define the list assignment L' by $L'(u) = \{d\}$ and $L'(v) = L(v)$ for $v \in G' - u$. By Lemma 4.1 there exists a path L' -coloring c of G' , and thus a path L -coloring of G . So Theorem 1.2 follows immediately from Lemma 4.1.

The proof of Lemma 4.1 describes a linear time path list-coloring algorithm, see Algorithm 4.2. For $v \in G$ and $u \in N(v)$, we will assume that $\text{next}(vu)$, $\text{prev}(vu)$, and vu can be computed in constant time from vu , see augmented adjacency lists in Section 2.

Algorithm 4.2: Path 3-Choose

Input. Let C be a cycle or length 1 path in a weakly triangulated plane graph G represented via augmented adjacency lists. Let $x, y, z \in C$ be such that $z \in C[x, y]$.

Let L be a length n array of lists of colors such that for $v \in \{x, y, z\}$ each list $L(v)$ has length one, for $v \in C - \{x, y, z\}$ each list $L(v)$ has length two or three, and for $v \in \text{int}(C)$ each list $L(v)$ has length three. Furthermore, for each $v \in C[x, z] - z$ assume that $L(v)$ does not contain the color in $L(z)$.

Let B, F be length n arrays such that for each $v \in C$ we have $B(v) = {}^v b$ and $F(v) = {}^v f$, where b is the vertex immediately counter clockwise from v around C , and f is the vertex immediately clockwise from v around C .

Let S be a length n integer array such that, for $v \in \overline{\text{int}}(C)$, $S(v) \neq 0$ if and only if $v \in C$, and $S(v) = S(x)$ if and only if $v \in C[x, z] - z$ (or $v = x = z$). Let $\max(S) = \max_{v \in G} S(v)$ and $m_C = m(\text{int}(C))$. If $z \neq x$, then define $\ell = \max(S)$. If $z = x$ and $C[y, x] \neq y, x$, then define $\ell = \max(S) + 1$. Otherwise, define $\ell = \max(S) + 2$. Assume that $\ell < 2(m - m_C) + 4$.

Let M be a length $2m + 2$ integer array such that if $v \in C$, then $M(S(v)) = S(y)$ if and only if $v \in C[z, y]$.

The concrete input will be G, L, B, F, S, M , and the tuple (x, y, z) .

Output. For each $v \in \overline{\text{int}}(C)$, all but one color will be removed from the list $L(v)$. The remaining color in each list will represent a path L -coloring of $\overline{\text{int}}(C)$ such that $\deg_c(x) \leq 1$, $\deg_c(y) \leq 1$, and $\deg_c(z) \leq 1$. If $z = y$, or if $C[z, y] = z, y$ and $L(z)$ did not contain the color in $L(y)$, then $\deg_c(z) = 0$. Moreover, afterwards $\max(S) \leq \ell + 2(m_C - 1)$.

Procedure. Assign ${}^zu_1 \leftarrow B(z)$. Our strategy will follow the proof of Lemma 4.1.

Base case. If $B(z) = F(z)$, then $C = z, u_1$ is a length 1 path. If $u_1 \neq x$ and $u_1 \neq y$, remove the color in $L(z)$ from $L(u_1)$, if it exists, then remove further colors until one color remains. Note that $m_C = 1$ and $\max(S) \leq \ell + 2(m_C - 1) = \ell$ by definition.

Recursive case. Suppose that $B(z) \neq F(z)$. Note that $n(C) > 2$ and therefore C is a cycle. Assign ${}^zu_2 \leftarrow \text{next}({}^zu_1)$. It follows immediately that $\max(S) \leq \ell + 2(m_C - 1)$ after Cases 1, 2, and 3.1, so we will only argue that this bound is satisfied after Case 3.2. It may be useful to refer to Figures 8, 9, and 10.

Case 1. Suppose that $u_1 = y$. Then $z = x$, since $z \in C[x, y]$. Set $S(x) \leftarrow \max(S) + 1$ and $S(y) \leftarrow S(x)$. Assign $M(S(x)) \leftarrow S(x)$. Make a recursive call with the input (y, x, y) .

Case 2. Suppose that $z = x$ and $u_1 \neq y$. Remove the color in $L(z)$ from $L(u_1)$, if it exists, then remove further colors until one color remains. Assign $S(u_1) \leftarrow \max(S) + 1$ and $M(S(u_1)) \leftarrow S(u_1)$, and make a recursive call with input (u_1, y, z) .

Case 3. Suppose that $z \neq x$ and $u_1 \neq y$. Remove zu_1 by assigning $F(u_1) \leftarrow \text{prev}({}^{u_1}z)$ and $B(z) \leftarrow \text{next}({}^zu_1)$; refer to Figure 8 (right), Figure 9, and Figure 10.

Case 3.1. Suppose that $S(u_2) = 0$, that is $u_2 \in \text{int}(C)$. Assign $S(u_2) \leftarrow S(x)$ and initialize $B(u_2) \leftarrow {}^{u_2}u_1 = \text{next}({}^{u_2}z)$ and $F(u_2) \leftarrow {}^{u_2}z$; refer to Figure 8 (right). Remove the color in $L(z)$ from $L(u_2)$. Make a recursive call with the same input (x, y, z) .

Case 3.2. Suppose that $S(u_2) \neq 0$, that is, $u_2 \in C$. Just as in Case 3.2 of the proof of Lemma 4.1, we will separately consider the two blocks G_1 and G_2 of $\overline{\text{int}}(C) - zu_1$. Note that $m_C - 1 = m(G_1) + m(G_2)$.

As we've already removed the edge zu_1 by adjusting $B(z)$ and $F(u_1)$, it remains to "split" the neighborhood of u_2 at the edge ${}^{u_2}z$ such that the recursive call on G_1 considers only the neighbors of u_2 counter-clockwise from z , and the call on G_2 considers z and all neighbors clockwise from z ; refer to Figure 9 and Figure 10.

Store ${}^{u_2}b \leftarrow B(u_2)$ and ${}^{u_2}f \leftarrow F(u_2)$. To represent G_1 , assign $B(u_2) \leftarrow \text{next}({}^{u_2}z)$ and $F(u_2) \leftarrow {}^{u_2}f$. To represent G_2 , assign $B(u_2) \leftarrow {}^{u_2}b$ and $F(u_2) \leftarrow {}^{u_2}z$.

Case 3.2.1. Suppose that $S(u_2) = S(x)$, i.e. $u_2 \in C[x, z]$. Make a recursive call on G_2 with input (x, y, z) ; afterwards $\max(S) \leq \ell + 2m(G_1) - 2$. Assign $S(u_2) \leftarrow \max(S) + 1$ and $M(S(u_2)) \leftarrow S(u_2)$, and make a recursive call on G_1 with input (u_2, u_2, u_2) . Finally, observe that $\max(S) \leq \ell + 2m(G_1) - 2 + 1 + 2m(G_2) - 1 < \ell + 2(m_C - 1)$.

Case 3.2.2. Suppose that $z \neq x$ and $S(u_2) \neq 0$, but $S(u_2) \neq S(x)$ and $M(S(u_2)) \neq S(y)$. In other words, suppose that $u_2 \in C[y, x] - x - y$. First remove the color in $L(z)$ from $L(u_2)$, if it exists, then remove colors until only one remains. Set $S(u_2) \leftarrow S(x)$. Make a recursive call on G_1 with input (x, u_2, x) ; afterwards $\max(S) \leq \ell + 2m(G_1)$. Set $S(u_2) \leftarrow \max(S) + 1$ and $M(S(u_2)) \leftarrow S(u_2)$, and make a recursive call on G_2 with input

(u_2, y, z) . Finally, observe that $\max(S) \leq \ell + 2m(G_1) + 1 + 2m(G_2) - 2 < \ell + 2(m_C - 1)$.

Case 3.2.3. Suppose that $z \neq x$ and $M(S(u_2)) = S(y)$, i.e. $u_2 \in C[z, y]$. There are two cases to consider, see Figure 10.

Case 3.2.3.1. Suppose that the color in $L(z)$ is in $L(u_2)$. Remove all colors from $L(u_2)$ except for the color in $L(z)$. Make a recursive call on G_1 with input (x, y, u_2) ; afterwards $\max(S) \leq \ell + 2m(G_1) - 2$. Assign $S(z) \leftarrow \max(S) + 1$, $S(u_2) \leftarrow S(z)$, and $M(S(z)) \leftarrow S(z)$. Make a recursive call on G_2 with input (u_2, z, u_2) . Finally, observe that $\max(S) \leq \ell + 2m(G_1) - 2 + 1 + 2m(G_2) - 1 < \ell + 2(m_C - 1)$.

Case 3.2.3.2. Suppose that the color in $L(z)$ is not in $L(u_2)$. Assign $M(S(x)) \leftarrow S(y)$. Make a recursive call on G_1 with input (x, y, x) ; afterwards $\max(S) \leq \ell + 2m(G_1)$. Assign $S(z) \leftarrow \max(S) + 1$, $S(u_2) \leftarrow S(z)$, and $M(S(z)) \leftarrow S(z)$. Make a recursive call on G_2 with input (u_2, z, u_2) . Observe $\max(S) \leq \ell + 2m(G_1) + 1 + 2m(G_2) - 1 = \ell + 2(m_C - 1)$.

Note that this is the only case where $M(S(v))$ is set a value other than $S(v)$. Setting $M(S(x)) \leftarrow S(y)$ effectively “joins” $C[x, u_1]$ onto $C[u_2, y]$ to represent $C_1[z_1, y_1]$ in G_1 , without having to re-mark each vertex in $C[x, u_1]$ with $S(y)$; see Figure 8.

Let G be a triangulated plane graph represented by augmented adjacency lists. Let L be a length n array of color lists of length 3, representing a list assignment of G . We may path L -color G using Algorithm 4.2 as follows.

Create length n arrays B, F and initialize $B(v)$ and $F(v)$ for each $v \in C$ on the outer face of G . Create a length n integer array S and initialize $S(v) \leftarrow 1$ for each $v \in C$ and $S(v) \leftarrow 0$ for each $v \in G - C$. Pick any $u \in C$, assign $S(u) \leftarrow 2$, and remove all but one color from $L(u)$. Create a length $2m + 2$ array of integers M and initialize $M(i) \leftarrow i$ for each $i \in \{0, 1, 2\}$. Finally, apply Algorithm 4.2 with the input G, L, B, F, S, M , and (u, u, u) . The remaining color in each list in L will represent a path L -coloring of G .

Each case in Algorithm 4.2 performs a fixed number of constant time operations. Case 3 removes an edge from the in the subgraph(s) under consideration. Case 1 makes a recursive call with an input that satisfies Case 2. Case 2 makes a recursive call with an input that satisfies Case 3. Therefore the total number of recursive calls made, and thus the overall time complexity, is $\mathcal{O}(n)$. See Figures 11, 12, and 13 for a concrete example.

5 Experimental Performance and Parallelism

A C implementation of each algorithm is available [3]. For each i from $3 \dots 7$, a uniform random sample of 100 plane triangulations of order 10^i was generated. Each algorithm was run on a each set of samples, see the timings in Figure 14 and the plot in Figure 16.

Plane triangulations were sampled using the method of Poulalhon and Schaeffer [10]. The vertices were randomly re-labeled in each sampled graph, and the corresponding adjacency list arrays were re-ordered in memory. The list assignments for Algorithm 4.2 were uniformly drawn from a set of six colors.

A breadth-first search (BFS) implemented with a circular array FIFO queue was run on the same set of graphs as a baseline algorithm that hits every half-edge of a graph exactly

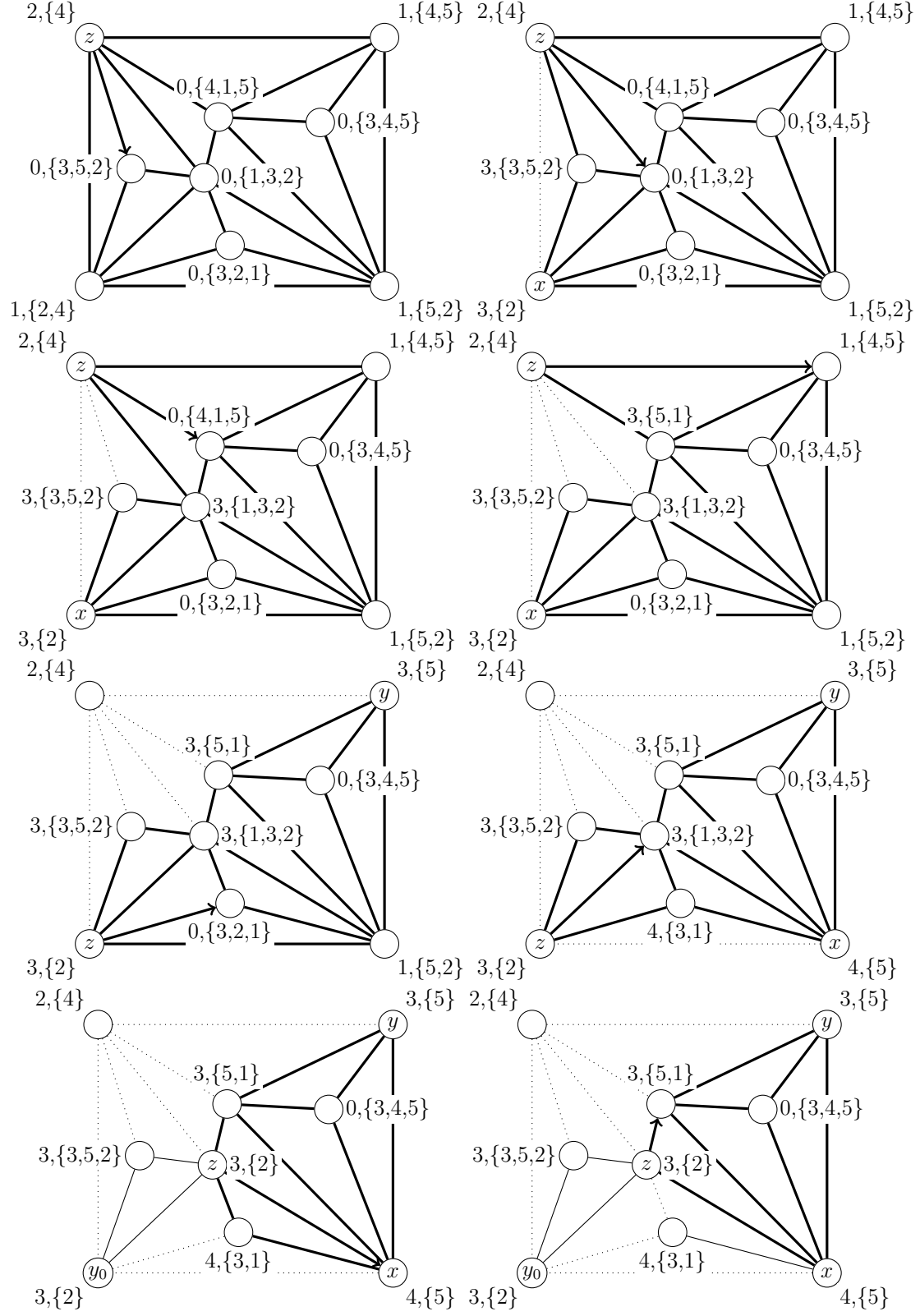


Figure 11: An example of Algorithm 4.2. Each vertex $v \in G$ is labeled with $S(v), L(v)$.

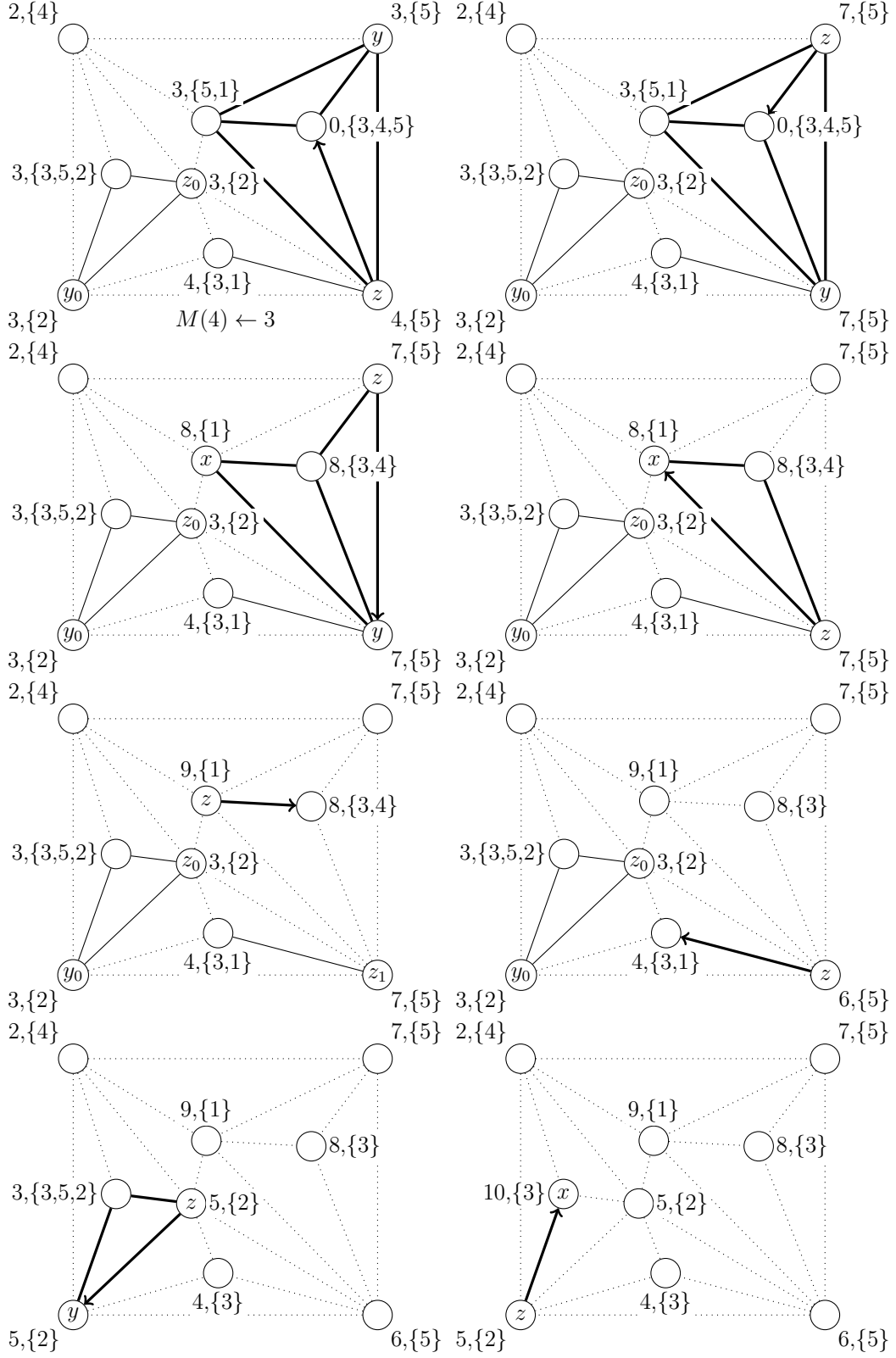


Figure 12: An Algorithm 4.2 example, continued from Figure 11.

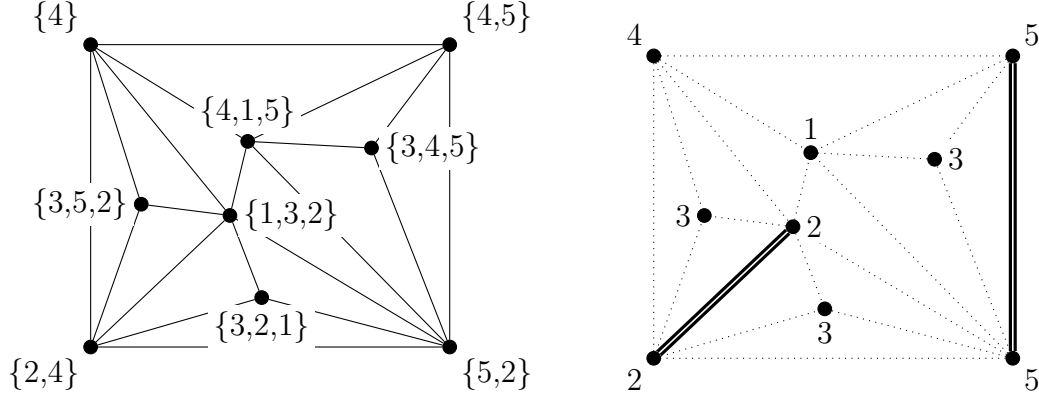


Figure 13: The initial list assignment L (left) and the induced paths of the resulting path L -coloring (right) from the Algorithm 4.2 example shown in Figure 11 and Figure 12.

once. All binaries were compiled with Clang 19.1.7 and run on a Linux 6.12.9 machine with an x86_64 Intel N100 processor. Both source code and binaries are available [3].

The performance of Algorithm 4.2 was similar to Algorithm 3.3 on the sample, but Algorithm 4.2 requires an augmented adjacency list representation. If one is not immediately available, then time required to execute Algorithm 4.2 would also include Algorithm 2.1.

A benchmark was also run to test the performance of Algorithm 3.2 and Algorithm 3.3 on the collection of pyramid graphs $\{A_k\}_{k \in \mathbb{N}}$ from Figure 3. Algorithm 3.2 scaled very poorly on pyramids, as expected; see the timings in Figure 15 and plot in Figure 16. Once again, vertex labels were uniformly shuffled, and adjacency lists re-ordered in memory. For each $n = 10^i$ we selected the closest integer k to $\sqrt{2n - 6}$ so that $n(A_k) \approx n$.

Algorithm 3.3 and Algorithm 4.2 perform recursive calls that operate on subgraphs that are disjoint apart from select vertices on their respective outer faces. Therefore it is possible to maintain a stack of independent recursive frames and have a pool of threads execute frames concurrently. The amount of parallelism possible with this method is highly dependent on the structure of the graph. E.g. when Algorithm 3.3 is run on a pyramid graph from Figure 3 there will never be more than one frame on the stack.

Our parallel implementation of Algorithm 3.3 maintains a shared lock-guarded stack from which all threads push and pop recursive frames. If no frames are available, a thread must idle until either a new frame is pushed to the stack by another thread or all threads are idle. To reduce lock contention, each thread may also maintain a small local stack of frames, and only access the shared stack when this local stack is full or empty. For the implementation and hardware used for benchmarks, the most optimal setup proved to be a local stack of 16 frames for each thread: if a local stack filled up, 8 frames were pushed to the shared stack; if a local stack emptied, 8 frames were popped from the shared stack.

Algorithm 4.2 is trickier to adapt to parallel execution. The first difficulty we encounter is that the recursive calls made in Algorithm 4.2 relied on a sequential execution order. If we save a local copy of $S(v)$, $B(v)$, $F(v)$ for each $v \in \{x, y, z\}$ alongside each frame, then

	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$	$n = 10^7$
(Black) BFS	$9.72 \cdot 10^3$	$2.82 \cdot 10^5$	$3.94 \cdot 10^6$	$1.04 \cdot 10^8$	$1.31 \cdot 10^9$
(Green) Algorithm 2.1	$3.35 \cdot 10^4$	$5.34 \cdot 10^5$	$2.10 \cdot 10^7$	$4.15 \cdot 10^8$	$4.65 \cdot 10^9$
(Brown) Algorithm 3.2	$2.24 \cdot 10^5$	$4.02 \cdot 10^6$	$6.78 \cdot 10^7$	$1.99 \cdot 10^9$	$3.57 \cdot 10^{10}$
(Red) Algorithm 3.3	$4.63 \cdot 10^4$	$5.99 \cdot 10^5$	$7.38 \cdot 10^6$	$1.58 \cdot 10^8$	$1.83 \cdot 10^9$
(Blue) Algorithm 4.2	$4.59 \cdot 10^4$	$6.46 \cdot 10^5$	$9.89 \cdot 10^6$	$1.84 \cdot 10^8$	$2.12 \cdot 10^9$

Figure 14: Average time (ns) per triangulated plane graph (sample size 100).

	$n \approx 10^3$	$n \approx 10^4$	$n \approx 10^5$	$n \approx 10^6$	$n \approx 10^7$
(Brown) Algorithm 3.2	$2.59 \cdot 10^5$	$8.71 \cdot 10^6$	$4.78 \cdot 10^8$	$7.38 \cdot 10^{10}$	$2.95 \cdot 10^{12}$
(Red) Algorithm 3.3	$1.95 \cdot 10^4$	$2.07 \cdot 10^5$	$3.49 \cdot 10^6$	$1.48 \cdot 10^8$	$2.26 \cdot 10^9$

Figure 15: Average time (ns) per pyramid graph from Figure 3 (sample size 30).

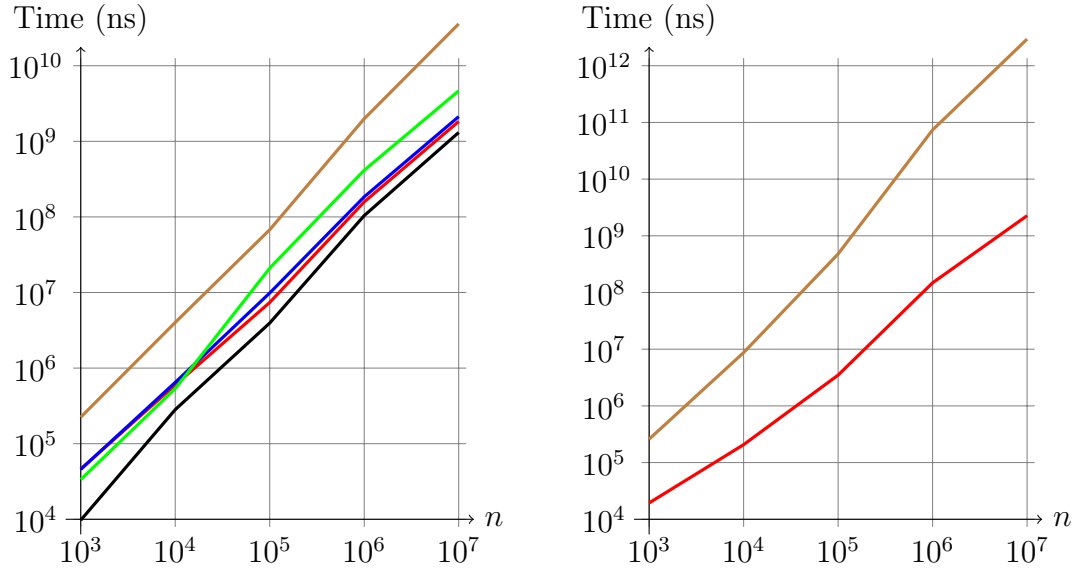


Figure 16: Plots of Figure 14 (left) and Figure 15 (right).

Threads	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$	$n = 10^7$
1	$4.63 \cdot 10^4$	$5.99 \cdot 10^5$	$7.38 \cdot 10^6$	$1.58 \cdot 10^8$	$1.83 \cdot 10^9$
2	$4.04 \cdot 10^4$	$4.61 \cdot 10^5$	$4.49 \cdot 10^6$	$9.04 \cdot 10^7$	$1.04 \cdot 10^9$
3	$4.12 \cdot 10^4$	$3.58 \cdot 10^5$	$3.50 \cdot 10^6$	$6.75 \cdot 10^7$	$7.83 \cdot 10^8$
4	$4.32 \cdot 10^4$	$3.14 \cdot 10^5$	$2.98 \cdot 10^6$	$5.59 \cdot 10^7$	$6.53 \cdot 10^8$

Figure 17: Parallel Algorithm 3.3 average time (ns) per graph (sample size 100).

Threads	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$	$n = 10^7$
1	$4.59 \cdot 10^4$	$6.46 \cdot 10^5$	$9.89 \cdot 10^6$	$1.84 \cdot 10^8$	$2.12 \cdot 10^9$
2	$6.22 \cdot 10^4$	$5.99 \cdot 10^5$	$6.97 \cdot 10^6$	$1.23 \cdot 10^8$	$1.40 \cdot 10^9$
3	$6.09 \cdot 10^4$	$5.02 \cdot 10^5$	$5.65 \cdot 10^6$	$9.32 \cdot 10^7$	$1.06 \cdot 10^9$
4	$6.58 \cdot 10^4$	$4.82 \cdot 10^5$	$5.15 \cdot 10^6$	$7.89 \cdot 10^7$	$8.90 \cdot 10^8$

Figure 18: Parallel Algorithm 4.2 average time (ns) per graph (sample size 100).

the recursive calls in Case 3.2.1 with $u_2 = x$, Case 3.2.2, Case 3.2.3.1, and Case 3.2.3.2 with $u_2 = y$ become order independent. An atomic counter can be used to track $\max(S) + 1$. Unfortunately, in Case 3.2.1 with $u_2 \neq x$ and Case 3.2.3.2 with $u_2 \neq y$, the second recursive call will always depend on the color list $L(u_2)$ produced by the first call.

Our parallel implementation of Algorithm 4.2 maintains a lock-guarded object pool of frames and a stack of references to frames in the pool. Recursive frames are added to the pool when they appear in the usual course of the algorithm, but are only pushed to the stack if they are ready to execute. For each vertex $v \in G$, store an optional reference to a frame that will become ready when v has been colored, i.e. has had its color list reduced to length 1. To allow for multiple frames to await the coloring of one vertex, each frame in the pool stores an “intrusive linked list,” that is, an optional reference to the next frame in line. When a frame is added to the pool and it needs to await a vertex that has another frame waiting on it, the new frame is added to the linked list of the existing frame. When a vertex v is colored, each frame waiting on v is pushed to the stack.

Benchmarks in Figures 17 and 18 showed some benefit to the parallel versions of Algorithms 3.3 and 4.2 when $n \geq 10^4$, with significant improvements for very large graphs.

References

- [1] J. Boyer and W. Myrvold, On the cutting edge: simplified $O(n)$ planarity by edge addition, *J. Graph Algorithms Appl.* **8** (2004), 241–273.
- [2] I. Broere and C. M. Mynhardt, Generalized colorings of outerplanar and planar graphs, *Graph theory with applications to algorithms and computer science* (Kalamazoo, Mich., 1984), pp. 151–161, Wiley-Intersci. Publ., Wiley, New York, 1985.
- [3] A. Bross, *Implementing path coloring algorithms on planar graphs*, Masters Project, University of Alaska, 2017, available from http://github.com/permutationlock/path_coloring_bg1.
- [4] G. G. Chappell and C. Hartman, Path choosability of planar graphs, *The Electronic Journal of Combinatorics* **25** (2018), P4.33.
- [5] G. Chartrand and H. V. Kronk, The point-arboricity of planar graphs, *J. London. Math. Soc.* **44** (1969), 612–616.

- [6] W. Goddard, Acyclic colorings of planar graphs, *Discrete Math.* **91** (1991), no. 1, 91–94.
- [7] C. M. Hartman, *Extremal Problems in Graph Theory*, Ph.D. Thesis, University of Illinois, 1997.
- [8] J.E. Hopcroft and R.E. Tarjan, Efficient Planarity Testing, *Journal of the ACM* **21** (1974), 549–568
- [9] K. S. Poh, On the linear vertex-arboricity of a planar graph, *J. Graph Theory* **14** (1990), no. 1, 73–75.
- [10] D. Poulalhon and G. Schaeffer, Optimal Coding and Sampling of Triangulations, *Algorithmica* **46** (2006), 505–527
- [11] R. Škrekovski, List improper colourings of planar graphs, *Combin. Probab. Comput.* **8** (1999), no. 3, 293–299.
- [12] D. B. West, *Introduction to Graph Theory*, 2nd ed., Prentice Hall, Upper Saddle River, NJ, 2000.